Reproducibility Study of WavesFM: A Multimodal Wireless Foundation Model

Local Benchmarks, Dataset Audits, and Cross-System Execution

Maheen Fatima
Department of Electrical and Software Engineering
University of Calgary, Canada

April 2026

Abstract

This report documents a reproducibility study of WavesFM, a multimodal wireless foundation model that uses a shared Vision Transformer backbone to process both raw IQ signals and image-like wireless data across multiple downstream tasks. Starting from the publicly released code, pretrained checkpoint, and dataset instructions, every preprocessing cache was rebuilt, the code was adapted for the available hardware, and the complete benchmark suite—a total of 126 individual training runs across three experimental sessions—was executed on a university lab GPU server. The results are encouraging but mixed. Eight of the ten attempted tasks reproduced closely: RF fingerprinting, modulation classification, 5G NR positioning, UWB indoor and industrial localization, RADCOM signal classification, interference classification, and DeepMIMO LoS/NLoS all fell within a fraction of a percentage point of the published numbers. Human activity sensing also came close, though with a caveat about the data splitting protocol. DeepMIMO beam prediction, however, showed a persistent 9–11 percentage point gap that survived three corrective attempts, pointing to an underspecified benchmark artifact that is not publicly available. One additional task, RF signal classification, could not be attempted because the publicly available raw data does not match the format the code expects.

Contents

1 Introduction **4**
- 1.1 The Problem: Too Many Models, Not Enough Reuse . . . 4
- 1.2 The Foundation Model Approach . . . 4
- 1.3 What WavesFM Is . . . 4
- 1.4 Why Verification Matters . . . 4

2 Background Concepts **5**
- 2.1 Wireless Signal Types . . . 5
- 2.2 Vision Transformers (ViT) . . . 5
- 2.3 Masked Autoencoders (MAE) . . . 6
- 2.4 Transfer Learning and Fine-Tuning . . . 6

3 WavesFM Model Architecture **7**
- 3.1 Two Phases: Pretraining and Fine-Tuning . . . 7
- 3.2 Model Size . . . 9
- 3.3 Fine-Tuning: Reusing the Pretrained Encoder . . . 10

4 Downstream Tasks and Evaluation Metrics **12**
- 4.1 Metrics . . . 13

5 Reproduction Methodology **14**
- 5.1 Starting Point . . . 14
- 5.2 Acquiring and Preprocessing the Datasets . . . 14
- 5.3 Hardware: MacBook Pro, Then Lab Server . . . 15
- 5.4 Code Modifications . . . 15
- 5.5 Training Configuration . . . 16
- 5.6 Three Experimental Sessions . . . 16

6 Official Baseline Results **17**

7 Reproduction Results **17**
- 7.1 The Full Picture . . . 17
- 7.2 Eight Tasks That Reproduced Successfully . . . 19
- 7.3 Human Activity Sensing: Close, With a Split Caveat . . . 27
- 7.4 DeepMIMO Beam Prediction: Not Reproduced . . . 28
  - 7.4.1 Three Attempts to Close the Gap . . . 29
  - 7.4.2 Why This Gap Exists . . . 29

8 Discussion **30**
- 8.1 Implications for the WavesFM Claims . . . 30
- 8.2 The Beam Prediction Problem . . . 30
- 8.3 The Role of Dataset Size . . . 30

8.4 Practical Considerations 31

9 Conclusion **31**

A Per-Seed Variability **33**

B Complete Code Change Log **33**

C Experimental Session Details **34**

1 Introduction

1.1 The Problem: Too Many Models, Not Enough Reuse

Modern wireless communication systems—the networks behind WiFi, 5G, Bluetooth, and emerging 6G—generate data at every stage of the signal processing chain. When a phone connects to a cell tower, the radio signals can be captured as raw waveforms, processed into frequency-domain representations called *spectrograms*, analyzed as *channel state information* (CSI) that describes how the signal traveled through the environment, or examined as *channel impulse responses* (CIR) that show how the signal bounced off surfaces over time.

Each of these data types is useful for different tasks. CSI can reveal what activity a person is performing in a room (sensing). Spectrograms can identify what kind of radio signal is being transmitted (classification). Raw waveforms can fingerprint which specific device sent a signal (identification). Channel data can estimate where a user is located (positioning).

Traditionally, each of these tasks requires its own specially designed machine learning model, its own training pipeline, and its own dataset. Building a WiFi sensing model teaches the system nothing about 5G positioning. A modulation classifier shares no knowledge with an interference detector. This fragmented approach is expensive: every new task starts from scratch.

1.2 The Foundation Model Approach

In other areas of artificial intelligence, a different paradigm has gained traction: *foundation models*. Instead of training a separate model for each task, a single model is trained on a broad variety of unlabeled data. This pretrained model learns general-purpose features—patterns and structures that are useful across many different tasks. For any specific task, the pretrained model is then adapted with a smaller amount of task-specific training, a process called *fine-tuning*. This is the approach behind well-known models like GPT (for language) and CLIP (for images).

WavesFM asks whether this same approach can work for wireless signals.

1.3 What WavesFM Is

WavesFM [1, 2] is a multimodal wireless foundation model developed by Ahmed Aboulfotouh and Hatem Abou-Zeid at the University of Calgary. It builds on the original WavesFM work [2], which introduced a ViT-based foundation model for image-like wireless modalities, and extends it to also handle raw IQ signals—the most fundamental form of radio data, capturing the signal's amplitude and phase over time. The project is publicly available at `https://wavesfm.waveslab.ai/` [3], where the authors have released the code, pretrained model weights, benchmark results, and reproduction instructions [4].

1.4 Why Verification Matters

The claims made by WavesFM are significant: if one model really can transfer across WiFi sensing, 5G positioning, device fingerprinting, interference detection, beam prediction, and more, that

would represent a meaningful step forward for wireless AI. But significant claims require careful verification. A model that performs well only in the original authors' environment, with their specific data splits and preprocessing choices, is less valuable than one whose results can be independently confirmed.

This report documents an independent attempt to reproduce the WavesFM benchmark results. The entire process is described—from understanding the model to acquiring datasets, fixing bugs in the released code, running experiments on two different machines, and comparing the results against the published numbers. Most results did reproduce, but with important exceptions that are analyzed in detail.

2 Background Concepts

This section introduces the key technical concepts needed to understand WavesFM. Readers already familiar with transformers, masked autoencoders, and wireless signal types may skip ahead to Section 3.

2.1 Wireless Signal Types

IQ signals. The most fundamental representation of a radio signal. "IQ" stands for In-phase and Quadrature—two components that together fully describe a signal's amplitude and phase at each moment in time. IQ data is captured directly by the radio hardware and contains the most information, but is also the hardest to work with because it is high-dimensional and noisy.

Spectrograms. A visual representation of how a signal's frequency content changes over time. A spectrogram has time on the horizontal axis, frequency on the vertical axis, and color or brightness representing signal strength. This format reveals patterns (like WiFi bursts or Bluetooth frequency hops) that are difficult to spot in raw IQ data.

Channel State Information (CSI). A measurement of how a wireless channel (the path between a transmitter and receiver) affects the signal. CSI captures effects like signal attenuation, phase shifts, and multipath propagation (where a signal bounces off walls and arrives at the receiver via multiple paths). CSI is particularly useful for sensing and positioning tasks because it encodes information about the physical environment.

Channel Impulse Response (CIR). Similar to CSI but in the time domain. A CIR shows how an ultra-short pulse would be received after traveling through the wireless channel, revealing distinct echoes from different objects and surfaces. CIR is commonly used in ultra-wideband (UWB) localization systems.

2.2 Vision Transformers (ViT)

A *transformer* is a neural network architecture, originally designed for language processing, that has since been applied successfully across many domains [7]. The key mechanism is *self-attention*:

the model learns which parts of its input are most relevant to each other, regardless of their position in the sequence.

A *Vision Transformer* (ViT) adapts this architecture to images. Instead of processing an image with convolutional filters, a ViT:

1. Divides the image into small, non-overlapping *patches* (e.g., 16×16 pixels each).
2. Converts each patch into a numerical vector (called a *token*) using a linear projection.
3. Adds *positional embeddings*—extra information encoding where each patch was located in the original image.
4. Processes all tokens through a series of *transformer blocks*, where self-attention allows every patch to attend to every other patch.

The output is a set of feature vectors that encode both local details and global context. WavesFM uses a ViT as its shared backbone, processing both image-like wireless data (by treating them as images) and IQ signals (by treating time-series segments as analogous to image patches).

2.3 Masked Autoencoders (MAE)

A *masked autoencoder* [8] is a self-supervised pretraining technique. The process works as follows:

1. Randomly hide (mask) a large fraction of the input—typically 70–75%.
2. Feed only the visible parts to the encoder.
3. Attempt to reconstruct the hidden parts using a lightweight decoder.

By learning to reconstruct missing portions, the encoder develops representations that capture the underlying structure of the data. After pretraining, the decoder is discarded and the encoder is reused for downstream tasks. This approach requires no labeled data—the "labels" are the masked portions of the input itself.

WavesFM extends this to the wireless domain with what the authors call *masked wireless modeling*: the same masked reconstruction objective applied simultaneously to both IQ streams and image-like wireless data.

2.4 Transfer Learning and Fine-Tuning

Transfer learning is the practice of taking a model trained on one task and reusing its learned features for a different task. The pretrained model has already learned general-purpose patterns; fine-tuning adapts these patterns to the specific requirements of the new task.

WavesFM evaluates three fine-tuning strategies, each using a different degree of adaptation:

Linear Probing (LP): The pretrained encoder is completely *frozen*—its weights are not updated at all. Only a small, freshly initialized output layer (the *task head*) is trained. This is the strictest test of how useful the pretrained features are without any adaptation.

Partial Fine-Tuning (FT2): Most of the encoder is frozen, but the last two transformer blocks are unfrozen and allowed to adapt. This provides some flexibility to adjust higher-level features while preserving the general features learned in earlier layers.

Low-Rank Adaptation (LoRA): Instead of modifying the encoder's weights directly, small trainable "adapter" matrices are inserted into the attention layers. The original weights remain frozen. This is a parameter-efficient approach—it achieves adaptation with roughly 5× fewer trainable parameters than FT2.

These three regimes form a spectrum from minimal to moderate adaptation. If a pretrained model performs well even with LP, it indicates that the learned representations are genuinely useful without any task-specific tuning. Section 3 shows how each of these modes operates inside the WavesFM architecture, with detailed diagrams.

3 WavesFM Model Architecture

3.1 Two Phases: Pretraining and Fine-Tuning

WavesFM operates in two distinct phases. In **Phase 1 (pretraining)**, the model learns general wireless knowledge from unlabeled data by reconstructing randomly masked inputs. In **Phase 2 (fine-tuning)**, the pretrained encoder is reused for specific downstream tasks with a small task-specific output layer attached on top. Figure 1 illustrates the pretraining process.

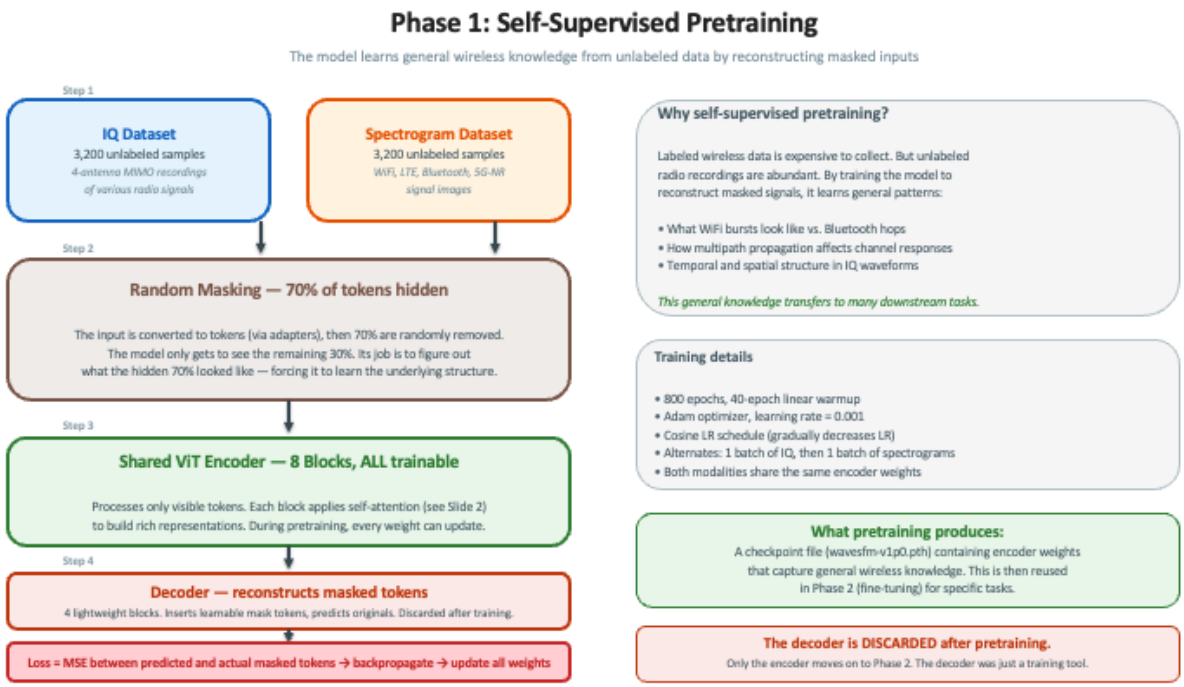


Figure 1: Phase 1: Self-supervised pretraining. Unlabeled IQ and spectrogram data are randomly masked (70% of tokens hidden). The encoder processes only visible tokens—with all weights trainable—and a lightweight decoder attempts to reconstruct the masked portions. The loss is mean squared error on masked positions. After 800 epochs, the decoder is discarded and only the encoder weights are kept, saved as the checkpoint file `wavesfm-v1p0.pth`.

The encoder is made up of 8 transformer blocks stacked in sequence. Each block performs three operations on every token: self-attention, a feed-forward network, and layer normalization. Figure 2 breaks down what happens inside a single block.

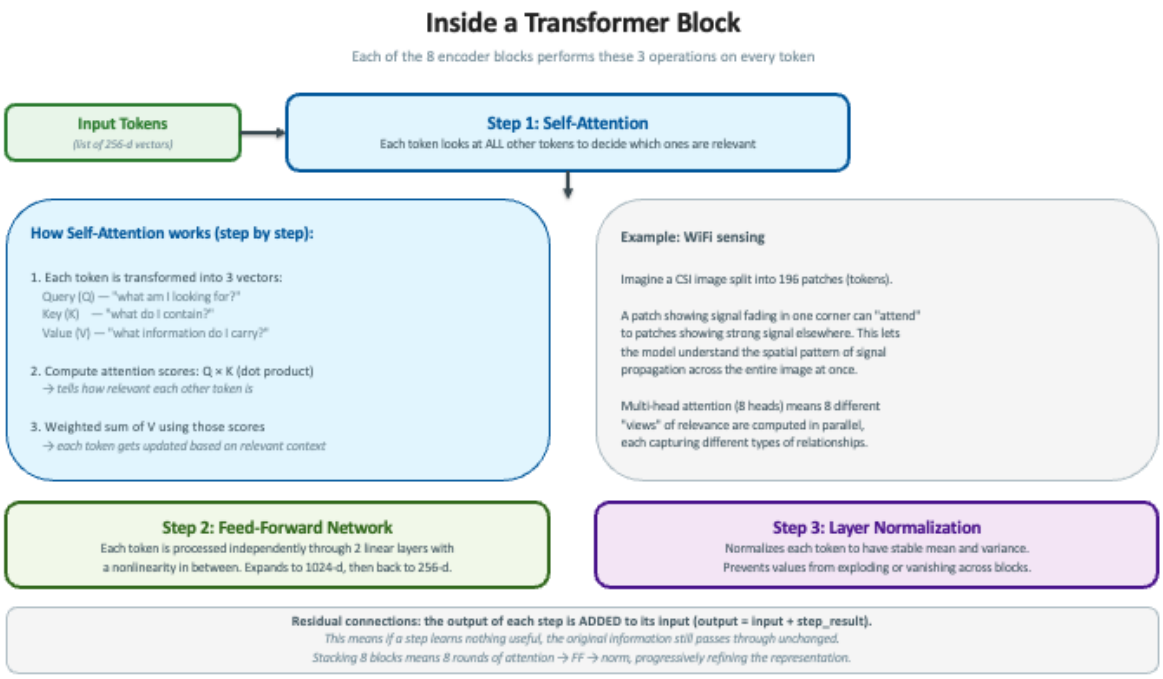


Figure 2: Inside a transformer block. Step 1: Self-attention—each token examines all other tokens to find relevant context (e.g., a token from one antenna attending to tokens from other antennas to understand spatial patterns). Step 2: A feed-forward network processes each token independently, expanding from 256 to 1024 dimensions and back. Step 3: Layer normalization stabilizes the values. Residual connections add each step's output to its input, ensuring information flows even if a particular step learns nothing useful.

The reconstruction loss that drives pretraining is mean squared error, computed only on the masked token positions:

$$\mathcal{L} = \frac{1}{|\mu_G|} \sum_{i \in \mu_G} \left\| \mathbf{x}_i^G - \hat{\mathbf{x}}_i^G \right\|^2 + \frac{1}{|\mu_Q|} \sum_{i \in \mu_Q} \left\| \mathbf{x}_i^Q - \hat{\mathbf{x}}_i^Q \right\|^2 \tag{1}$$

where μ_G and μ_Q are the sets of masked token indices for image-like and IQ inputs, respectively. Pretraining runs for 800 epochs with a 40-epoch warmup, using Adam optimizer with learning rate 10^{-3} and cosine annealing. The pretraining data consists of 3,200 spectrogram samples and 3,200 IQ samples [1].

3.2 Model Size

All published benchmarks use the **Small** configuration, summarized in Table 1. At roughly 7 million parameters total, this is a compact model. For context, the standard ViT-Base model from the original ViT paper [7] has 86 million parameters, and ViT-Large has 307 million—meaning WavesFM's encoder is roughly 12× smaller than ViT-Base.

Table 1: WavesFM model configurations. All benchmarks use the Small variant.

	Embed Dim	Blocks	Heads	MLP Ratio	Params (M)
Micro	128	4	4	4.0	—
Small	**256**	**8**	**8**	**4.0**	**6.32 (enc) + 0.79 (dec)**
Base	512	12	8	4.0	—
Large	768	12	12	4.0	—

3.3 Fine-Tuning: Reusing the Pretrained Encoder

Once pretraining is complete, the encoder's learned weights are loaded and reused for specific downstream tasks. The fine-tuning pipeline chains together the same modality adapters and conditioning layer from pretraining, but now adds a task-specific head on top of the encoder. Figure 3 shows this data flow end to end.

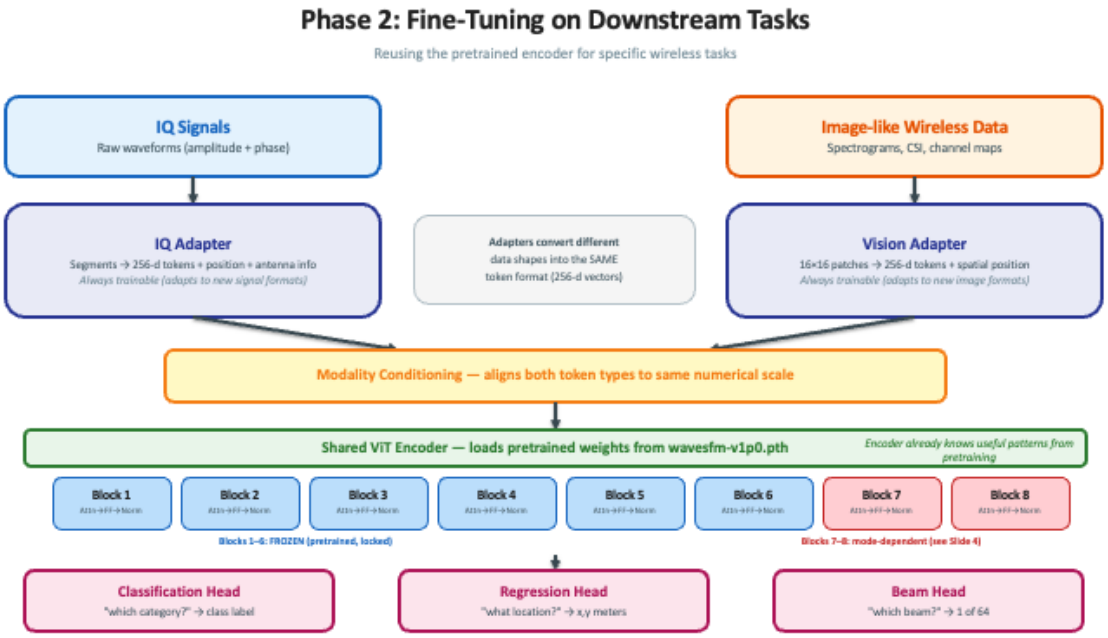


Figure 3: Phase 2: Fine-tuning data flow. IQ signals and image-like data each pass through their respective adapters (which convert different data shapes into the same 256-d token format), then a modality conditioning layer, then the shared ViT encoder loaded with pretrained weights. Task-specific heads—classification, regression, or beam prediction—produce the final output.

The key question during fine-tuning is: how much of the pretrained encoder should be allowed to change? Allowing too much change risks losing the general knowledge learned during pretraining. Allowing too little may prevent the model from adapting to the new task. WavesFM evaluates three strategies, compared side by side in Figure 4.

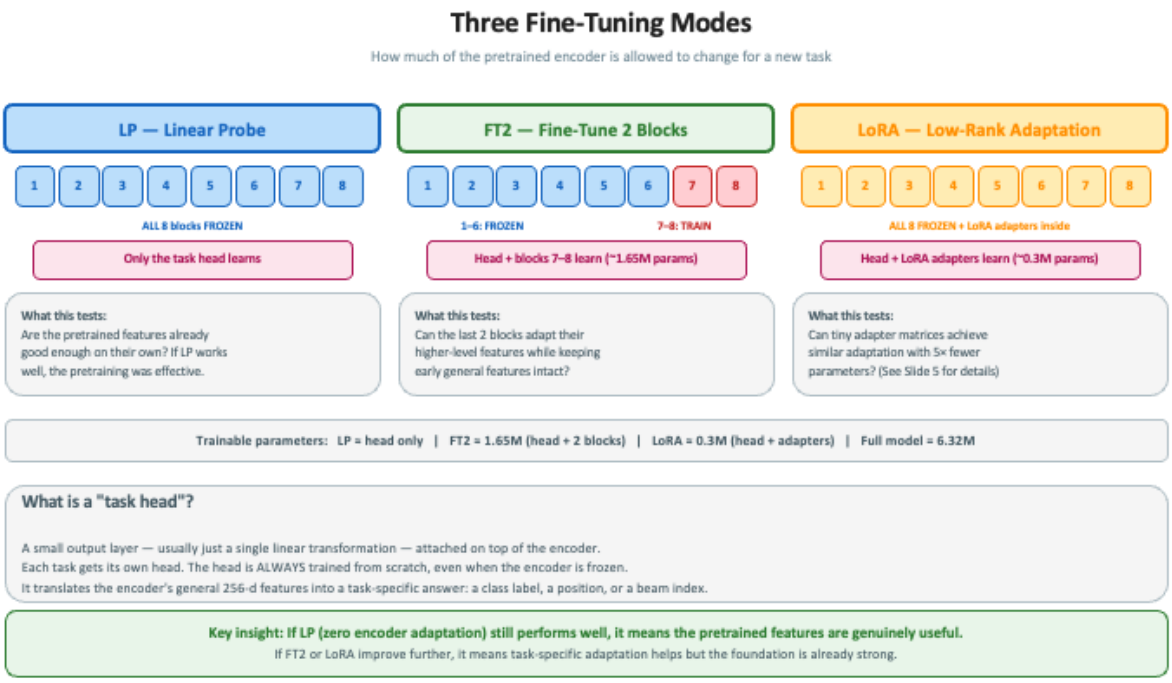


Figure 4: The three fine-tuning modes. LP (Linear Probe) freezes all 8 encoder blocks and trains only the task head—testing whether the pretrained features are already useful. FT2 (Fine-Tune 2) freezes blocks 1–6 and allows blocks 7–8 to adapt ($\sim$1.65M trainable parameters). LoRA (Low-Rank Adaptation) keeps all blocks frozen but inserts small adapter matrices inside each ($\sim$0.3M parameters). The bottom section explains what a "task head" is: a single linear layer that translates the encoder's general features into a task-specific answer.

Of the three modes, LoRA deserves special attention because of how it achieves adaptation without modifying the original weights. Instead of updating the full 256×256 weight matrices inside each attention layer (65,536 parameters each), LoRA inserts two small matrices that squeeze the signal through a low-dimensional bottleneck. Figure 5 walks through this mechanism step by step.

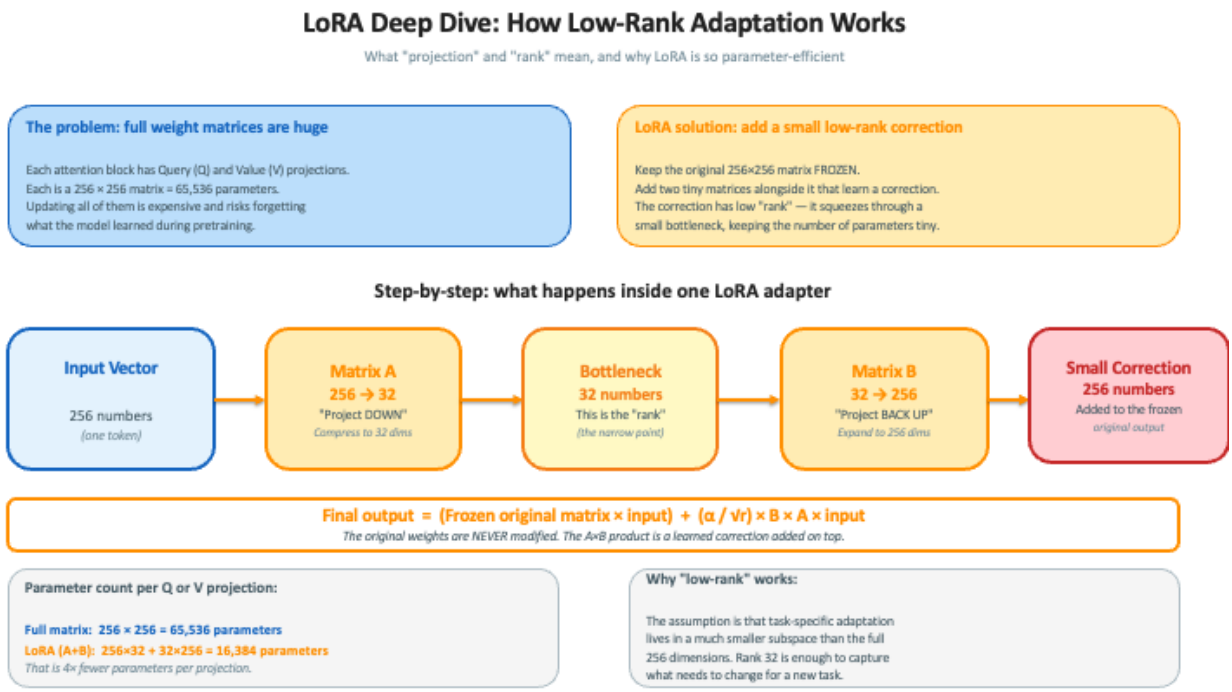


Figure 5: LoRA deep dive. The original weight matrix stays frozen. Two small matrices are added alongside: Matrix A projects the 256-dimensional input down to 32 dimensions (the "rank"), and Matrix B projects it back up to 256. The result is a small correction added to the frozen output. This requires only 16,384 parameters per projection instead of 65,536—a 4× reduction—yet achieves comparable adaptation to partial fine-tuning.

4 Downstream Tasks and Evaluation Metrics

The WavesFM benchmark evaluates the pretrained model on 11 downstream tasks, listed in Table 2. This breadth is deliberate: a foundation model should work across many different problems, not just one.

Table 2: The downstream tasks in the WavesFM benchmark, spanning two modality families and multiple application domains.

Task ID	Modality	What It Does	Classes	Metric
`sensing`	Vision (CSI)	Recognizes human activities (walking, sitting, lying down, etc.) from WiFi signals	6	PCA
`rfs`	Vision (spectrogram)	Classifies radio signal types (WiFi, FM, cellular, Bluetooth, etc.) [6]	20	PCA
`pos`	Vision (CSI)	Estimates a device's physical location from 5G channel measurements	—	Mean error
`radcom`	IQ	Classifies modulation and signal type from over-the-air captures	9	PCA
`uwb-indoor`	IQ (CIR)	Locates objects indoors using ultra-wideband radio pulses	—	Mean error
`uwb-industrial`	IQ (CIR)	Locates objects in industrial environments using UWB	—	Mean error
`rml`	IQ	Identifies modulation type (QPSK, QAM, etc.) from raw signals	11	PCA
`rfp`	IQ	Identifies which specific device transmitted a signal (RF fingerprinting)	4	PCA
`interf`	IQ	Detects and classifies radio interference	3	PCA
`deepmimo-los`	Vision (channel)	Determines if a signal has a direct path (line-of-sight) or not	2	PCA
`deepmimo-beam`	Vision (channel)	Predicts the best antenna beam direction for communication	64	PCA

4.1 Metrics

Per-Class Accuracy (PCA). The primary metric for all classification tasks. PCA computes accuracy separately for each class and then averages across classes. This is more informative than regular accuracy when some classes have far more samples than others, because it treats every class as equally important. For example, if a dataset has 90% of samples in class A and 10% in class B, a model that always predicts class A would achieve 90% regular accuracy but only 50% PCA.

Mean Distance Error. The primary metric for positioning tasks. It measures the average Euclidean distance (in meters) between the model's predicted location and the true location. Lower values indicate better performance.

Checkpoint Selection. During training, the model is evaluated on a held-out validation set after each epoch. The checkpoint with the best validation metric is kept as the final model—highest PCA

for classification, lowest mean error for regression.

5 Reproduction Methodology

5.1 Starting Point

The reproduction began with the publicly available artifacts from the WavesFM project:

- The code repository, containing the model definition, training scripts, and preprocessing pipelines
- The pretrained checkpoint (`wavesfm-v1p0.pth`), containing the model weights after self-supervised pretraining
- Dataset documentation on the WavesFM website [3], with links to raw data sources
- The official reproduce page [4], recommending Python 3.10+, PyTorch 2.x with CUDA, and a GPU comparable to an NVIDIA A100

5.2 Acquiring and Preprocessing the Datasets

Each task requires a specific raw dataset that must be downloaded from its public source and converted into an HDF5 cache file (a compact binary format optimized for fast loading during training). The WavesFM repository includes preprocessing scripts for each task. Table 3 shows the acquisition status.

Table 3: Dataset acquisition status for each task.

Task	Raw Data Source	Status
`sensing`	NTU-Fi HAR WiFi CSI `.mat` files	Ready
`pos`	IEEE Dataport 5G NR CSI measurements	Ready
`radcom`	RadCom OTA HDF5 signal captures	Ready
`rml`	RadioML 2022 modulation samples	Ready
`rfp`	POWDER PAWR platform IQ fingerprints	Ready
`uwb-indoor`	UWB indoor CIR data (4 environments)	Ready (after bug fix)
`uwb-industrial`	Industrial UWB localization data	Ready
`interf`	ICARUS interference IQ recordings	Ready
`deepmimo`	DeepMIMO scenario channel data via DeepMIMOv3	Ready (rebuilt twice)
`lwm-beam`	LWM beam-challenge pickle files	Ready (hypothesis only)
`rfs`	CommRad archive (`.wav` recordings)	**Blocked**—format mismatch

The `rfs` task could not proceed because the publicly available data (nested folders of `.wav` audio files labeled by radio device) does not match what the preprocessing script expects (a flat folder of spectrogram images labeled by signal type). The label semantics are fundamentally different, so no faithful reproduction was possible.

5.3 Hardware: MacBook Pro, Then Lab Server

Development started on a **MacBook Pro**, which was practical for code inspection, dataset acquisition, preprocessing, and documentation. The released code assumed NVIDIA CUDA hardware, so it was adapted for Apple's MPS GPU backend. Even with this adaptation, training was too slow for the full benchmark—10 tasks $\times$ 3 fine-tuning modes $\times$ 3 random seeds = 90 individual training runs.

The benchmarking was therefore moved to a **university lab Linux server** (`ngwn06`):

- **GPU:** NVIDIA GeForce RTX 5090
- **CPU:** 24 cores, 62 GB RAM
- **Software:** PyTorch 2.9.1, CUDA 12.4, timm 1.0.11, h5py 3.15.1

The MacBook remained the canonical editing environment. Code changes were made locally and synced to the server via `rsync`; results were synced back after each session. Long-running experiments were launched in `tmux` sessions to continue unattended.

5.4 Code Modifications

Running the released code required several modifications, categorized by impact:

Bug fixes (the code would not run without these).

- One script imported a module that does not exist in the release, crashing at startup.
- The UWB indoor preprocessing script referenced a command-line argument that was never defined.

Hardware adaptations (no effect on scientific results).

- Replaced hard-coded `cuda` device with automatic selection (CUDA $\rightarrow$ Apple MPS $\rightarrow$ CPU).
- Restricted mixed-precision training features to CUDA only (unsupported on Apple GPUs).
- Added crash-recovery hardening: atomic checkpoint saves, RNG state preservation, and fsynced logging.

Benchmark protocol changes (may affect results for specific tasks).

- Changed the DeepMIMO beam codebook from 16 to 64 beams to match the public WavesFM setup.
- Fixed a bug in the beam angle sweep that duplicated one endpoint and left the highest beam class unreachable (changed from inclusive $[-90°, +90°]$ to half-open $[-90°, 90°)$).
- Adjusted the model's output layer to match the full 64-beam codebook rather than the number of labels observed in the data.

- Added a separate experimental task (`lwm-beam-challenge`) for hypothesis testing, kept isolated from the official benchmark pipeline.

A full code change log is provided in Appendix B.

5.5 Training Configuration

Every task was trained using the hyperparameters specified in the WavesFM benchmark configuration, summarized in Table 4. All tasks shared the same optimizer (AdamW with weight decay 0.05), learning rate schedule (cosine annealing from 10^{-3} down to 10^{-6}, with a 5-epoch warmup), and layer-wise learning rate decay (factor 0.75).

Table 4: Task-specific training hyperparameters.

Task	Batch Size	Epochs	Label Smoothing	Notes
`sensing`	256	100	0.10	Stratified split
`pos`	256	100	—	—
`radcom`	2048	50	—	—
`uwb-indoor`	256	50	—	—
`uwb-industrial`	512	50	—	—
`rml`	2048	50	—	—
`rfp`	256	10	0.10	—
`interf`	256	35	0.02	Stratified split, class weights
`deepmimo-los`	256	100	—	Stratified split, class weights
`deepmimo-beam`	256	100	—	Stratified split, class weights

Each task/mode combination was run with three different random seeds (0, 1, 2) and the results averaged, following the official protocol. This ensures that results reflect genuine model behavior rather than a lucky initialization.

5.6 Three Experimental Sessions

Three sessions were conducted, each building on findings from the previous one:

1. **First full run** (April 3, 2026): All 10 available tasks, 3 modes each, 3 seeds = **90 runs**. This provided baseline results for every task.

2. **Corrected second pass** (April 15, 2026): Reran `sensing`, `deepmimo-los`, and `deepmimo-beam` after discovering and fixing the beam codebook bug. **27 runs**. These replaced the first-pass results for those three tasks.

3. **Hypothesis test** (April 15, 2026): Ran the `lwm-beam-challenge` experiment to investigate the beam prediction discrepancy. **9 runs**. This was a diagnostic experiment, not a benchmark reproduction.

Each session produced training logs, model checkpoints, summary statistics, local-vs-official comparison files, and visualization plots (confusion matrices for classification tasks, error density plots for positioning tasks).

6 Official Baseline Results

The official results were obtained from the WavesFM detailed evaluation page [5] and stored locally as a JSON snapshot (dated January 23, 2026). These are the target values the reproduction aimed to match.

Table 5: Official WavesFM classification results (PCA %, higher is better).

Task	LP	FT2	LoRA
`sensing`	94.42	99.31	98.47
`rfs`	42.85	83.41	84.49
`radcom`	90.10	94.53	93.78
`rml`	50.39	55.16	56.49
`rfp`	98.97	99.70	99.82
`interf`	71.50	78.94	78.94
`deepmimo-los`	95.02	95.46	95.24
`deepmimo-beam`	67.67	78.38	77.38

Table 6: Official WavesFM regression results (mean distance error in meters, lower is better).

Task	LP	FT2	LoRA
`pos`	3.17	1.24	1.45
`uwb-indoor`	1.43	0.83	0.65
`uwb-industrial`	2.71	0.88	0.74

7 Reproduction Results

7.1 The Full Picture

Table 7 presents all 30 task/mode results alongside the official baselines. For `sensing`, `deepmimo-los`, and `deepmimo-beam`, the numbers come from the corrected second pass; for all other tasks, they come from the first full run.

Table 7: Complete reproduction results. **Blue** = official published values; **green** = local reproduction. Classification: PCA (%); regression: mean error in meters. "Δ" = local − official (for classification, positive means the local result was higher; for regression, negative means the local error was lower). Superscripts: [1]first run, [2]corrected second pass.

		Linear Probe (LP)			Fine-Tune 2 (FT2)			LoRA			
Task	**Unit**	**Official**	**Local**	**Δ**	**Official**	**Local**	**Δ**	**Official**	**Local**	**Δ**	**Verdict**
`rfp`[1]	%	98.97	98.98	+0.01	99.70	99.70	≈0	99.82	99.84	+0.02	Reproduced
`rml`[1]	%	50.39	50.39	≈0	55.16	55.14	−0.02	56.49	56.42	−0.07	Reproduced
`radcom`[1]	%	90.10	90.12	+0.02	94.53	94.61	+0.08	93.78	94.07	+0.29	Reproduced
`interf`[1]	%	71.50	71.17	−0.33	78.94	78.83	−0.11	78.94	79.67	+0.73	Reproduced
`d.mimo-los`[2]	%	95.02	95.12	+0.10	95.46	95.36	−0.10	95.24	95.16	−0.08	Reproduced
`pos`[1]	m	3.17	3.14	−0.03	1.24	1.26	+0.02	1.45	1.51	+0.06	Reproduced
`uwb-in`[1]	m	1.43	1.44	+0.01	0.83	0.82	−0.01	0.65	0.65	≈0	Reproduced
`uwb-ind`[1]	m	2.71	2.71	≈0	0.88	0.88	≈0	0.74	0.74	≈0	Reproduced
`sensing`[2]	%	94.42	93.89	−0.53	99.31	98.47	−0.84	98.47	99.31	+0.84	Close (caveat)
`d.mimo-beam`[2]	%	67.67	56.37	−11.30	78.38	69.44	−8.94	77.38	67.84	−9.54	**Not reproduced**
`rfs`	%	42.85	—	—	83.41	—	—	84.49	—	—	**Blocked**

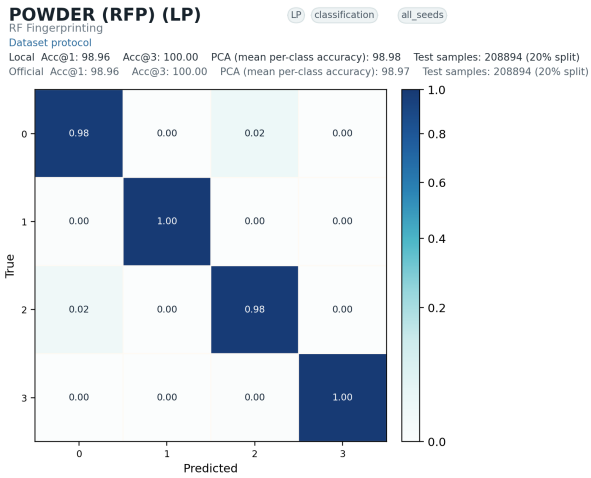
7.2 Eight Tasks That Reproduced Successfully

The strongest finding of this study is that **eight of ten attempted tasks reproduced within a fraction of a percentage point**. These span both modality families (vision and IQ) and both task types (classification and regression), confirming that the pretrained checkpoint transfers across diverse wireless problems.

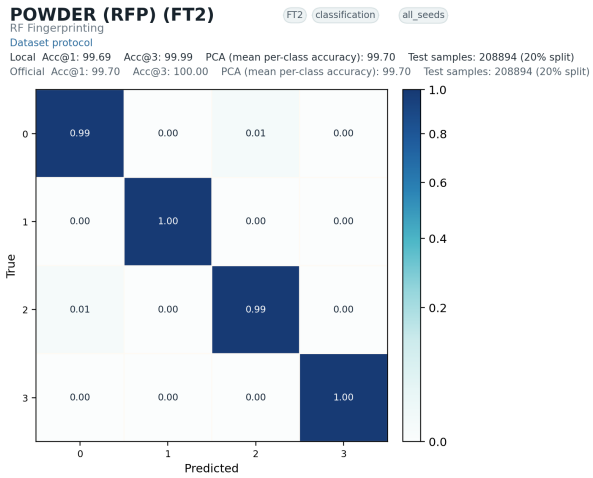
Notable results:

- **RF Fingerprinting**: The LoRA result (99.84%) differs from official (99.82%) by 0.02 percentage points. The standard deviation across seeds was only 0.01%, making this the most precisely reproduced task.
- **Modulation Classification**: The LP result matched the official value to three decimal places (50.393% vs. 50.39%).
- **UWB Industrial**: All three modes matched within 0.004 meters—effectively identical.
- **5G NR Positioning**: All modes within 0.06 meters of official values.
- **DeepMIMO LoS/NLoS**: All modes within 0.10 percentage points after the corrected second pass. This is important because the same dataset and preprocessor are used for the beam prediction task (Section 7.4), demonstrating that the pipeline works correctly.

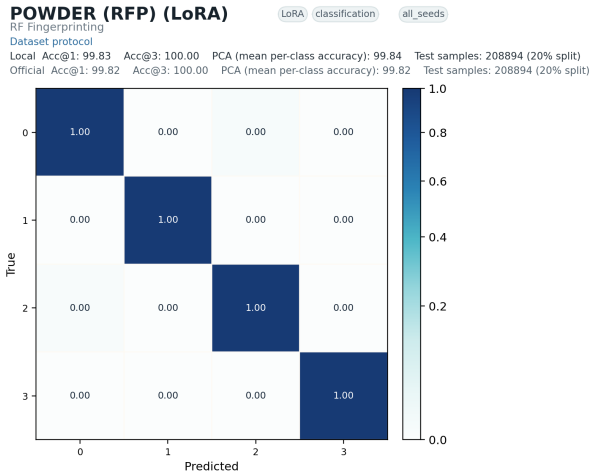
How to read the plots below: Each plot header contains two rows of numbers. The row labeled "**Local**" shows the **reproduced metrics** and the row labeled "**Official**" shows the **published baseline metrics**. Comparing these two rows shows how close the reproduction is to the original.
The **confusion matrix** or **error density curve** below the header visualizes the local reproduction in detail. In confusion matrices, diagonal cells show correct predictions (darker = more correct). In density curves, the shaded area shows the error distribution and the dashed red line marks the mean error. Official visualizations were not available—only the official metric values in the header.



(a) RFP – LP

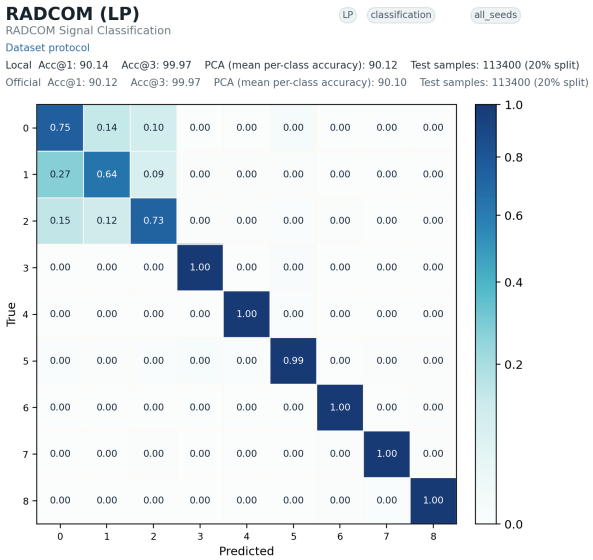


(b) RFP – FT2

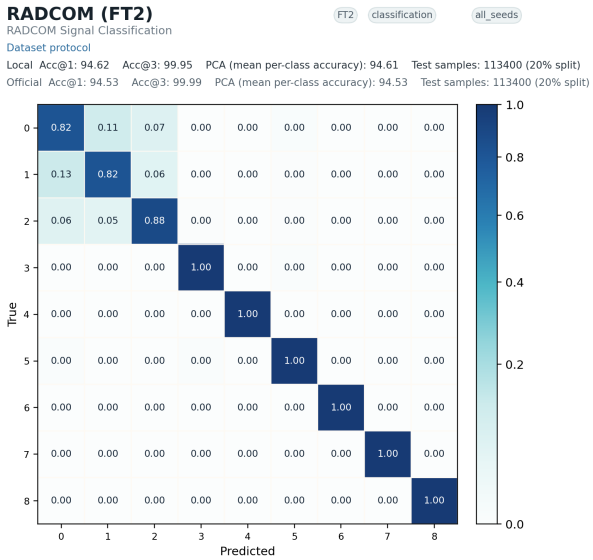


(c) RFP – LoRA

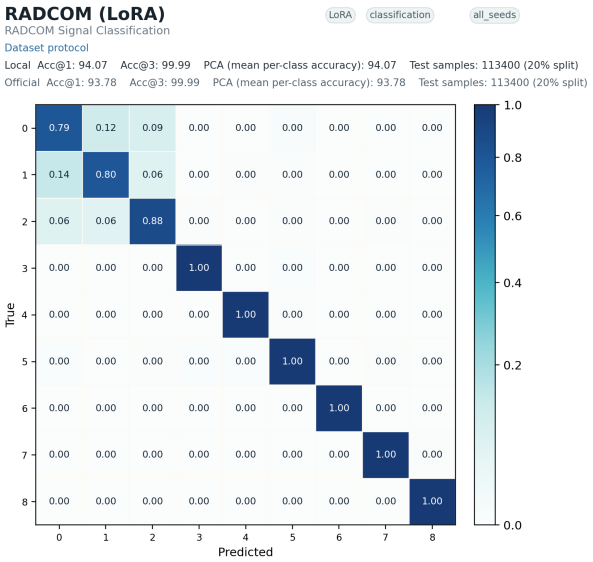
Figure 6: Confusion matrices for RF Fingerprinting (4 device classes). The near-perfect diagonal reflects >99% PCA across all modes.



(a) RADCOM – LP

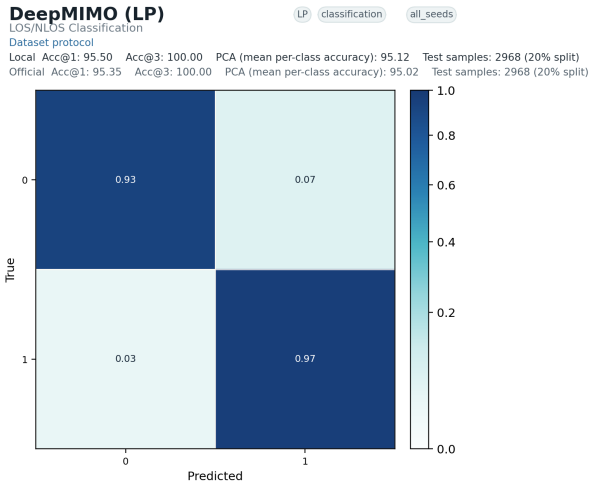


(b) RADCOM – FT2

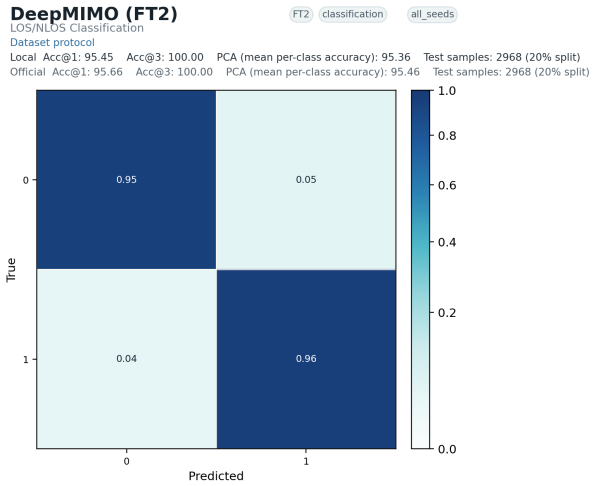


(c) RADCOM – LoRA

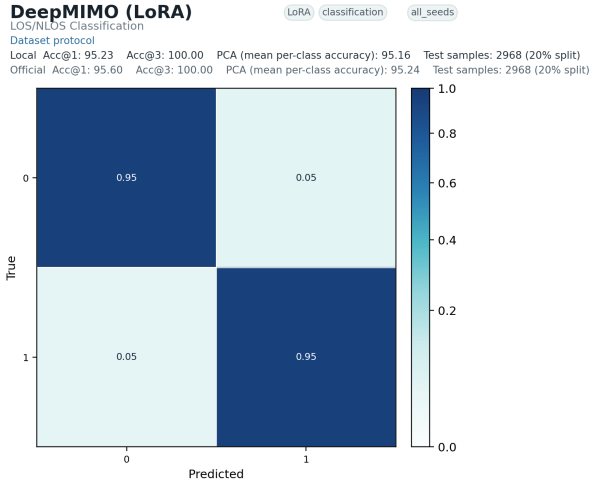
Figure 7: Confusion matrices for RADCOM signal classification (9 classes). Off-diagonal entries indicate misclassifications between similar signal types.



(a) LP

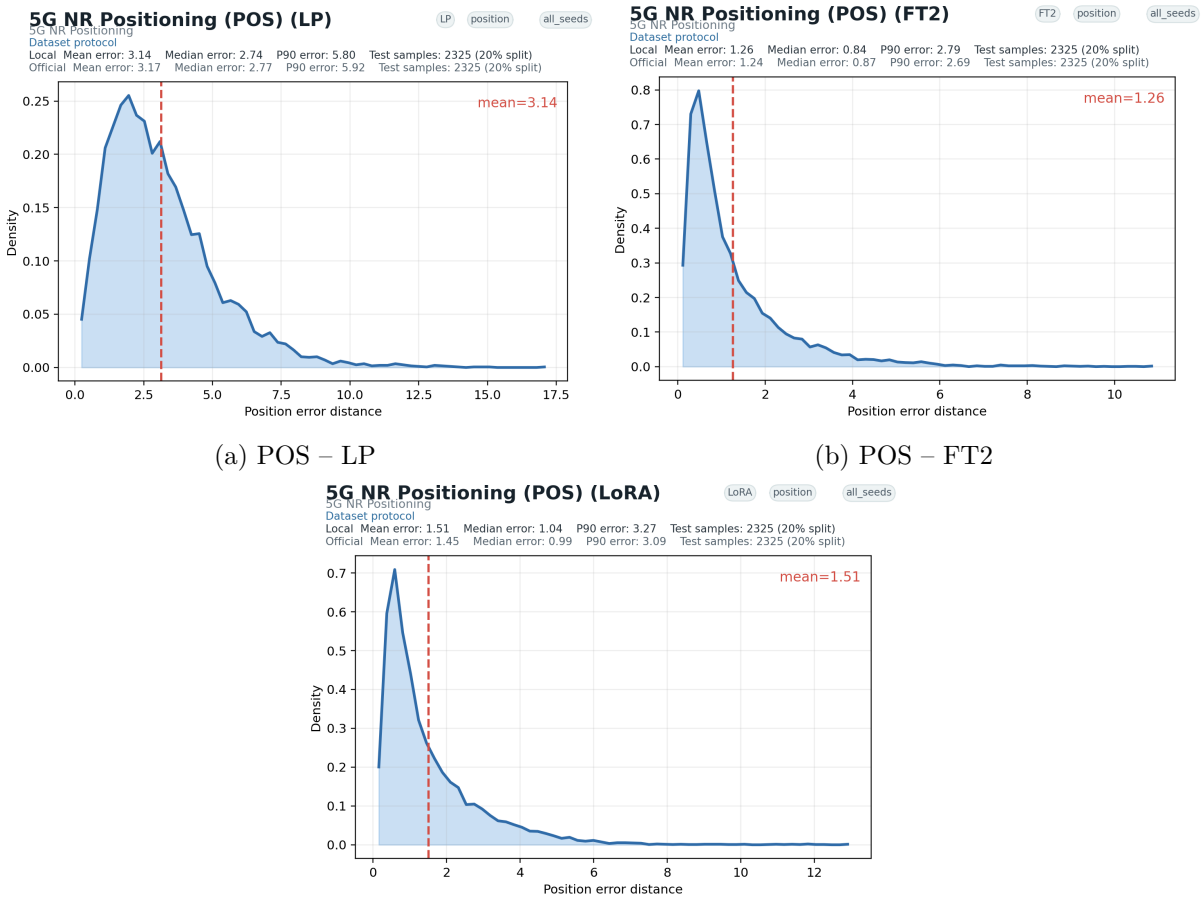


(b) FT2



(c) LoRA

Figure 8: Confusion matrices for DeepMIMO LoS/NLoS classification (2 classes). All modes fell within 0.10 percentage points of the official values.

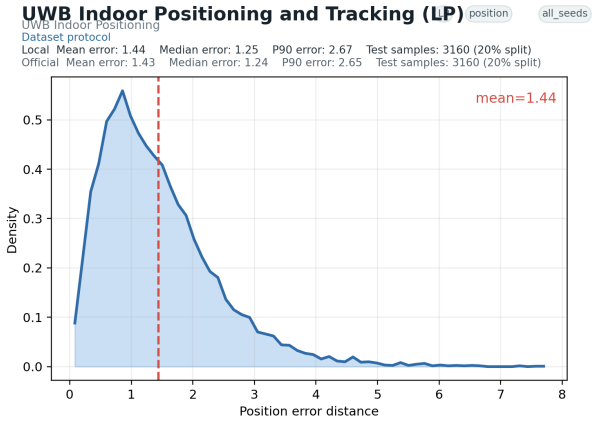


(a) POS – LP

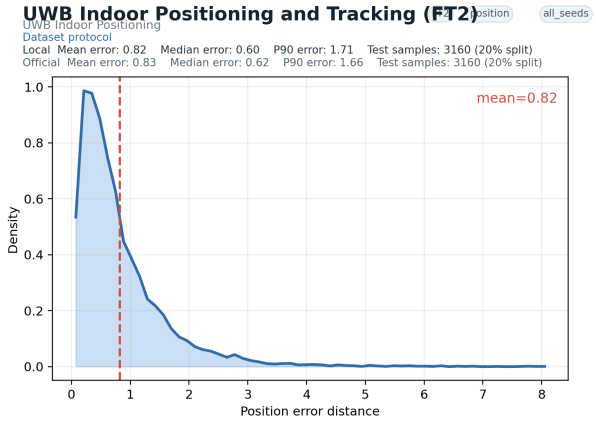
(b) POS – FT2

(c) POS – LoRA

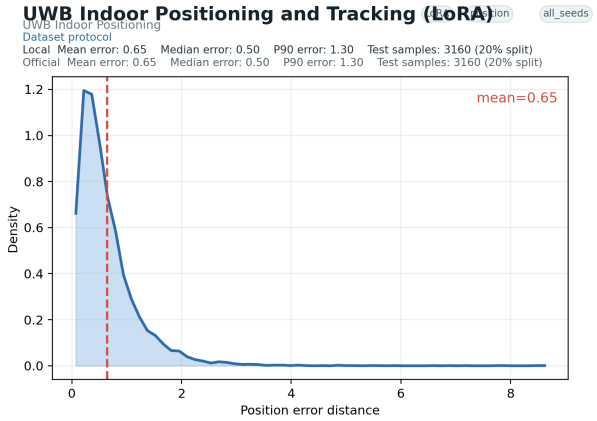
Figure 9: Position error density for 5G NR Positioning. A curve concentrated near zero indicates accurate localization. The shift from LP to FT2/LoRA shows clear improvement.



(a) UWB Indoor – LP

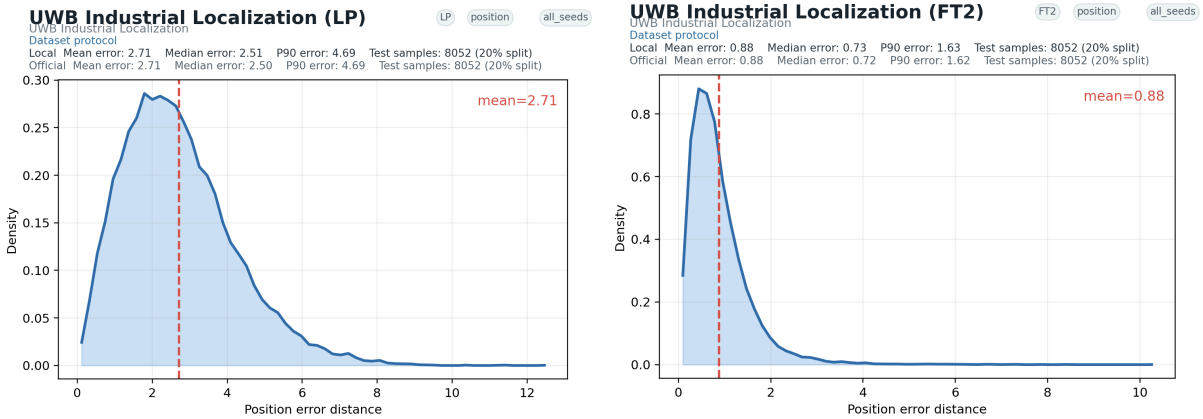


(b) UWB Indoor – FT2



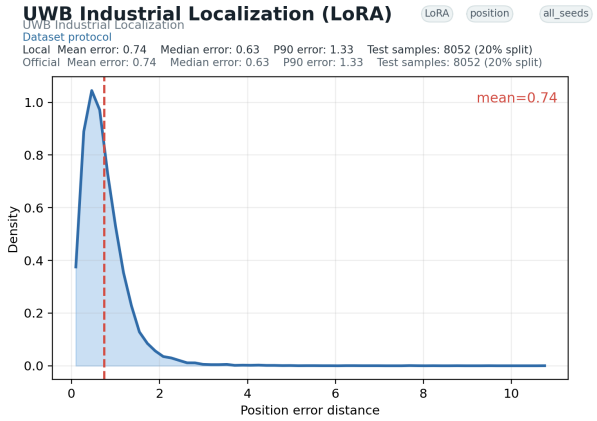
(c) UWB Indoor – LoRA

Figure 10: Position error density for UWB Indoor localization.



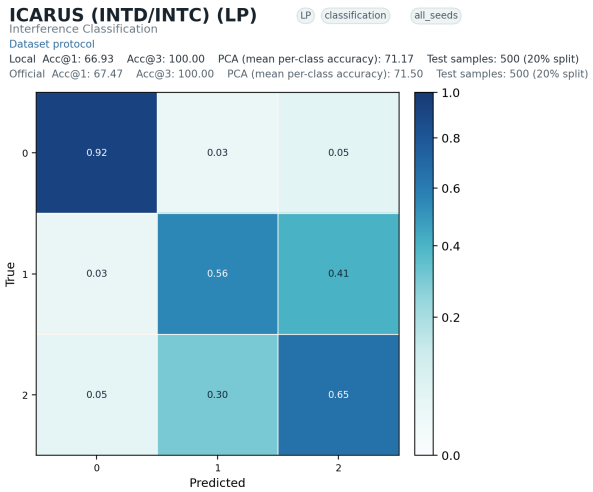
(a) UWB Industrial – LP

(b) UWB Industrial – FT2

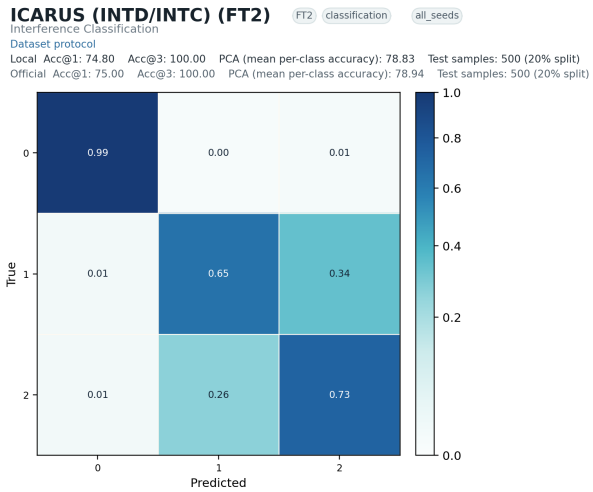


(c) UWB Industrial – LoRA

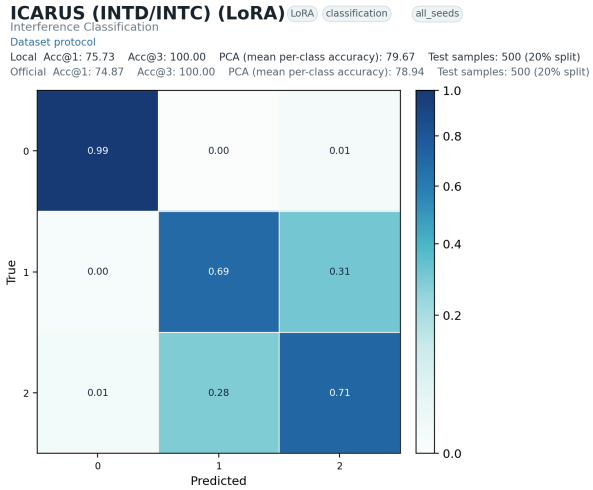
Figure 11: Position error density for UWB Industrial localization.



(a) Interference – LP



(b) Interference – FT2



(c) Interference – LoRA

Figure 12: Confusion matrices for ICARUS interference classification (3 classes).

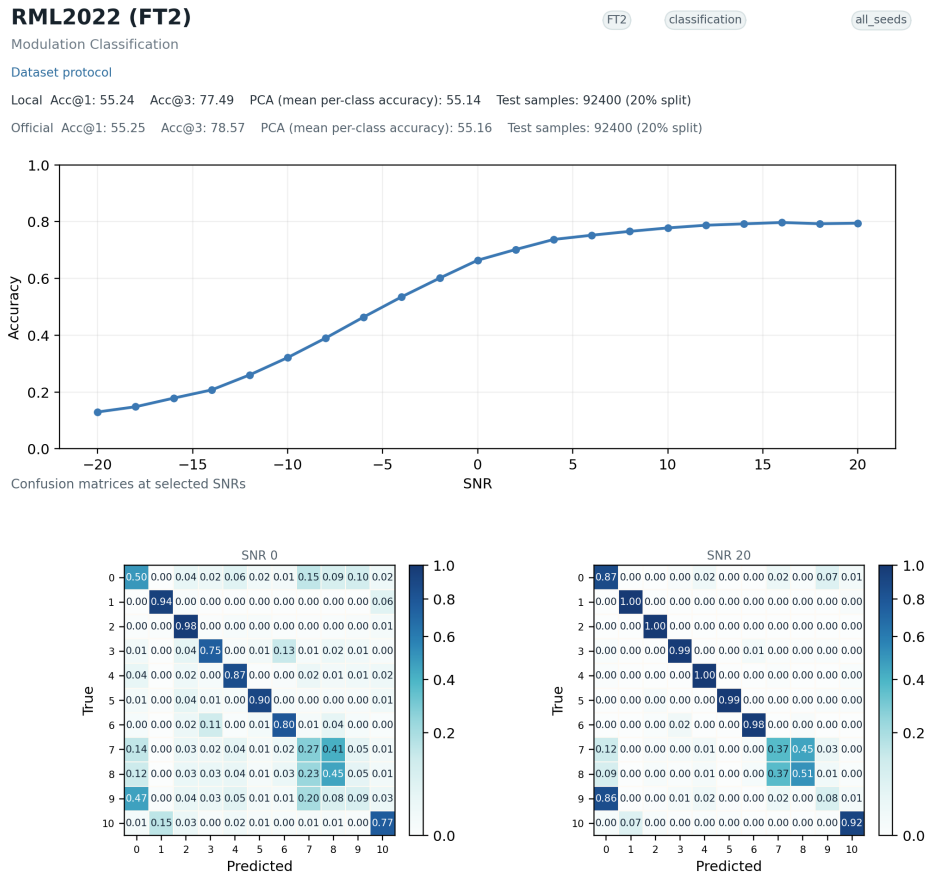


Figure 13: RML modulation classification overview for FT2, showing per-class performance across 11 modulation types.

7.3 Human Activity Sensing: Close, With a Split Caveat

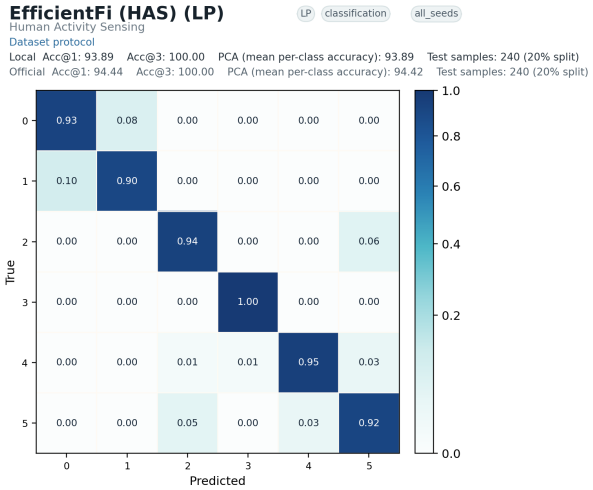
Sensing results were close to official values (within about 1 percentage point) but showed higher variance across seeds, partly due to the small test set (only 240 samples).

Table 8: Human activity sensing results (PCA %, corrected second pass). "Std" is the standard deviation across 3 random seeds.

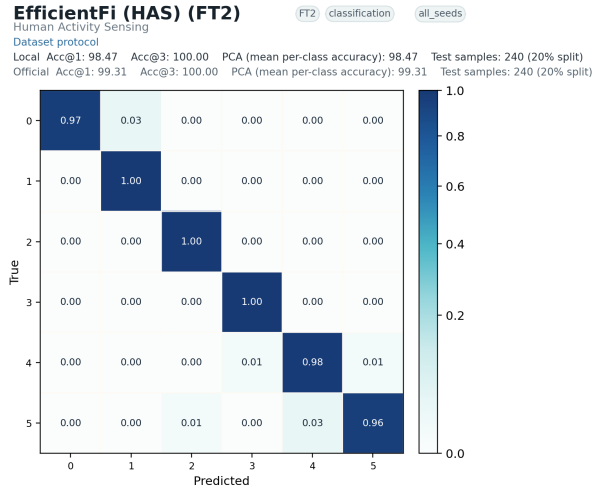
Mode	Official	Local (Mean)	Std	Δ
LP	94.42	93.89	1.99	−0.53
FT2	99.31	98.47	0.86	−0.84
LoRA	98.47	99.31	0.71	+0.84

The caveat involves the *data split protocol*—how the data is divided into training and validation sets. The WavesFM preprocessing code merges the original dataset's `train` and `test` directories into one pool and applies a new random 80/20 split. The exact split indices used for the published numbers are not available. The second pass used a deterministic stratified split, which comes close but cannot be guaranteed to match exactly. With only 240 test samples, even small split differences

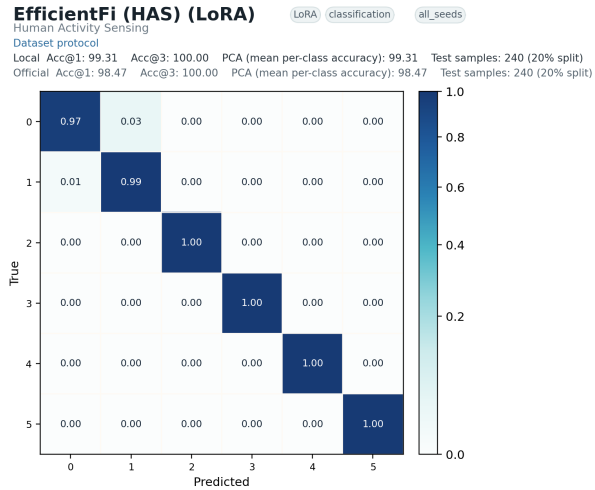
can shift PCA by 1–2 percentage points.



(a) LP



(b) FT2



(c) LoRA

Figure 14: Confusion matrices for human activity sensing (6 activity classes).

7.4 DeepMIMO Beam Prediction: Not Reproduced

This is the most significant negative finding. Beam prediction showed a persistent gap of 9–11 percentage points below official results, and the gap was highly consistent across seeds (standard deviation ≤ 0.64, far smaller than the gap itself).

Table 9: DeepMIMO beam prediction results (PCA %, corrected second pass). The large gaps (Δ) highlighted in red show the persistent discrepancy.

Mode	Official	Local (Mean)	Std	Δ
LP	67.67	56.37	0.64	−11.30
FT2	78.38	69.44	0.15	−8.94
LoRA	77.38	67.84	0.42	−9.54

Note: Confusion matrix plots for the 64-class beam prediction task are omitted because a 64×64 *grid is too dense to read at any practical print size. The numerical comparison in Table 9 above clearly shows the gap.*

7.4.1 Three Attempts to Close the Gap

Attempt 1: First pass (discarded). Used a pickle-sourced cache that turned out to have only 63 usable beam classes instead of 64. A bug in the beam angle sweep duplicated one endpoint, leaving the top class unreachable. These results were invalidated.

Attempt 2: Corrected second pass. Rebuilt the cache from official DeepMIMO scenario folders with the beam sweep bug fixed. Verified all 64 beam classes were present. The gap shrank slightly but remained at 9–11 percentage points.

Attempt 3: Alternative data source hypothesis. Tested whether the official numbers might have been produced from a different data source—the public LWM beam-challenge files (622 labeled samples). Results were near-random (PCA $\approx$ 1.7%), conclusively ruling out this hypothesis.

Table 10: LWM beam-challenge hypothesis test (PCA %). Official beam reference vs. LWM result. Near-random performance rules out this data source.

Mode	Beam Ref	LWM Result	Std	Δ
LP	67.67	1.88	0.37	−65.79
FT2	78.38	1.68	0.07	−76.70
LoRA	77.38	1.69	0.18	−75.69

The near-random PCA values (around 1.7% on a 64-class problem, where random chance is $100/64 \approx 1.6\%$) indicate the model learned nothing useful from this data source.

7.4.2 Why This Gap Exists

The gap is *not* caused by:

- **Random seed variation**—the standard deviation across seeds is $\leq 0.64\%$, while the gap is $\geq 8.94\%$.
- **The 63-class bug**—this was fixed before the second pass.

- **A different data source**—the LWM hypothesis was conclusively rejected.
- **A broken pipeline**—the same code and preprocessor reproduced DeepMIMO LoS/NLoS within 0.10 pp.

The most likely explanation is a *benchmark protocol mismatch*: the exact preprocessed data cache, the label-to-beam mapping, or the train/test split indices used for the official evaluation are not recoverable from the publicly available artifacts. The locally reconstructed cache contains 14,840 samples of shape $(2, 32, 32)$, but without access to the exact official cache or split, the gap cannot be closed. This is a **public artifact availability issue**, not a bug in the reproduction pipeline.

8 Discussion

8.1 Implications for the WavesFM Claims

The reproduction results largely support WavesFM's central claim—that a single multimodal ViT backbone can transfer to diverse wireless tasks. Eight tasks reproduced closely, spanning IQ-based tasks (fingerprinting, modulation recognition, interference detection, UWB localization), vision-based tasks (positioning, LoS/NLoS, RADCOM classification), and both classification and regression.

The linear probing results are particularly telling. Under LP, the encoder is completely frozen—no adaptation occurs. If the pretrained features were only useful for one narrow problem, LP would fail on everything else. Instead, LP performed respectably across the board, indicating that self-supervised pretraining on wireless data does produce broadly transferable representations.

8.2 The Beam Prediction Problem

The beam prediction failure is notable from a reproducibility standpoint. It is not a case of broken code—the same codebase reproduced 8 other tasks and even reproduced the other DeepMIMO task (LoS/NLoS) from the same dataset. The issue is narrowly scoped to the beam prediction label and split protocol.

This pattern—where the code and model are functional but one specific benchmark cannot be reconstructed from public artifacts—is a recognized challenge in machine learning reproducibility. It underlines why publishing code alone is insufficient for full reproducibility; exact preprocessed data caches and split indices are also needed.

8.3 The Role of Dataset Size

A clear pattern emerged: tasks with large test sets (RFP: 208,894 samples; RML: 92,400; RADCOM: 113,400) reproduced with high precision, while tasks with small test sets (sensing: 240; interference: 500) showed more variability. This is statistically expected—smaller samples produce noisier metric estimates—but it has practical implications. Publishing exact split indices becomes especially important for small-dataset tasks, where minor split differences can shift results meaningfully.

8.4 Practical Considerations

For researchers considering WavesFM as a starting point:

- The pretrained checkpoint transfers effectively across diverse wireless tasks.
- The code needs minor fixes before first use (two bug fixes, plus hardware adaptations if not using CUDA).
- Most benchmark numbers are reproducible from the public artifacts.
- The beam prediction gap is specific to the DeepMIMO beam task and does not undermine the broader model.
- The full benchmark suite (90+ runs) requires a capable GPU and some form of automated job management.

9 Conclusion

This reproducibility study of WavesFM yielded the following findings:

Successfully reproduced (8 tasks). RF fingerprinting, modulation classification, 5G NR positioning, UWB indoor and industrial localization, RADCOM signal classification, ICARUS interference classification, and DeepMIMO LoS/NLoS all reproduced within a fraction of a percentage point. These span both modality families and both task types, confirming that the WavesFM pretrained backbone transfers across diverse wireless problems.

Closely reproduced with caveat (1 task). Human activity sensing reproduced within approximately 1 percentage point, but the exact data split used for the official numbers is not published, creating an inherent verification gap on this small dataset.

Not reproduced (1 task). DeepMIMO beam prediction showed a persistent 9–11 percentage point gap across three experimental attempts. The most likely cause is an unpublished benchmark artifact (data cache or split indices).

Blocked (1 task). RF signal classification could not be attempted because the publicly available CommRad data format does not match what the preprocessing code expects.

Overall assessment. WavesFM is a genuine contribution to wireless AI. The multimodal foundation model concept is well-motivated, the architecture functions as described, and the majority of the benchmark reproduces closely. Three additions would achieve full reproducibility: (1) the exact preprocessed HDF5 caches used for official evaluation, (2) the exact train/validation split indices for all tasks, and (3) a tested end-to-end reproduction script that runs without requiring bug fixes.

References

[1] A. Aboulfotouh and H. Abou-Zeid, "Multimodal wireless foundation models," *arXiv:2511.15162v2*, Feb. 2026.
https://arxiv.org/abs/2511.15162

[2] A. Aboulfotouh, E. Mohammed, and H. Abou-Zeid, "6G WavesFM: A foundation model for sensing, communication, and localization," *IEEE Open J. Commun. Soc.*, vol. 6, 2025.
https://arxiv.org/abs/2504.14100

[3] WavesFM project homepage — code, pretrained weights, benchmark results, dataset documentation, and reproduction instructions.
https://wavesfm.waveslab.ai/

[4] WavesFM reproduction instructions.
https://wavesfm.waveslab.ai/docs/reproduce/

[5] WavesFM detailed evaluation results (official baseline numbers used in this study).
https://wavesfm.waveslab.ai/detailed-eval/

[6] M. Zahid, "CommRad RF: A dataset of communication radio signals for detection, identification and classification," Zenodo, 2024.
https://zenodo.org/records/14192970

[7] A. Dosovitskiy *et al.*, "An image is worth 16x16 words: Transformers for image recognition at scale," in *Proc. ICLR*, 2021.
https://arxiv.org/abs/2010.11929

[8] K. He *et al.*, "Masked autoencoders are scalable vision learners," in *Proc. IEEE/CVF CVPR*, 2022.
https://arxiv.org/abs/2111.06377

A Per-Seed Variability

Table 11 shows how much results varied across the three random seeds (0, 1, 2). Low standard deviation indicates stable, reliable results; high standard deviation suggests sensitivity to random factors.

Table 11: Standard deviation across 3 seeds for all task/mode combinations.

Task	**LP Std**	**FT2 Std**	**LoRA Std**
`rfp` (PCA %)	0.016	0.015	0.012
`rml` (PCA %)	0.096	0.166	0.084
`radcom` (PCA %)	0.052	0.052	0.319
`interf` (PCA %)	1.225	1.381	0.624
`deepmimo-los` (PCA %)	0.610	0.390	0.393
`sensing` (PCA %)	1.993	0.856	0.708
`deepmimo-beam` (PCA %)	0.640	0.147	0.418
`pos` (meters)	0.036	0.084	0.068
`uwb-indoor` (meters)	0.019	0.006	0.016
`uwb-industrial` (meters)	0.010	0.010	0.007

Tasks with large test sets (`rfp`, `rml`, `radcom`) show very low variance. Tasks with small test sets (`sensing`: 240 samples, `interf`: 500 samples) show higher variance, as expected.

B Complete Code Change Log

All modifications to the released `wavesfm/` codebase, by file:

1. `run_finetune_all.py`: Removed broken import; changed default beams 16→64; added `sensing` to stratified split set.
2. `main_finetune.py`: Device auto-selection; AMP restricted to CUDA; default beams 16→64; atomic checkpoint writes; RNG state save/restore; safe DataLoader tuning.
3. `utils.py`: NumPy seeding; fsynced JSONL logging; worker-seeding helpers.
4. `preprocess_uwb_loc.py`: Removed non-existent `args.root` reference (bug fix).
5. `preprocess_deepmimo.py`: Half-open beam sweep fix; pickle input support; cache audit attributes; full-length class weights.
6. `preprocess_radcom.py`: Single-pass statistics accumulation (same output).
7. `preprocess_csi_sensing.py`: Source-split provenance tracking; cache version v1→v2.
8. `preprocess_lwm_beam_challenge.py`: New preprocessor for hypothesis test.
9. `data.py`: Beam head sized from codebook; added `lwm-beam-challenge` task.
10. `dataset_classes/deepmimo.py`: Beam cache validation; malformed-cache warnings.
11. `requirements.txt`: Added `matplotlib==3.10.7`.

C Experimental Session Details

First full run `20260403_004122-ngwn06`

10 tasks × 3 modes × 3 seeds = 90 runs. Host: `ngwn06` (RTX 5090, 24 cores, 62 GB RAM). All 90 completed. Final results for 7 tasks.

Corrected second pass `second_try_sensing_deepmimo_20260415_005633-ngwn06`

3 tasks × 3 modes × 3 seeds = 27 runs. Same host. All 27 completed. Final results for `sensing`, `deepmimo-los`, and `deepmimo-beam`.

Hypothesis test `third_try_lwm_beam_challenge_20260415_140720-ngwn06`

1 task × 3 modes × 3 seeds = 9 runs. Same host. All 9 completed. Diagnostic only—conclusive negative result.